RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23331 - DESIGN AND ANALYSIS OF ALGORITHMS

Laboratory Record Note Book

Name	Suvanitha Sri S
Year / Branch / Section	2nd year CSE G
University Register No.	2116250701766
College Roll No.	250701766
Semester	3
Academic Year	2025-29

INDEX PAGE

Exp.No	Name of the Experiment	Date of Completion
1	Finding complexity using counter method	16/07/2026
2	Finding complexity using counter method	16/07/2026
3	Finding complexity using counter method	23/07/2026
4	Finding complexity using counter method	23/07/2026
5	Finding complexity using counter method	23/07/2026
6	1-G Coin problem	30/07/2026
7	2-G Cookies problem	30/07/2026
8	3-G Burger problem	06/08/2026
9	4-G Array sum max	06/08/2026
10	5-G Product of array elements-minimum	13/08/2026
11	1-Number of zeroes in a given array	13/08/2026
12	2-Majority element	20/08/2026
13	3-Finding floor values	20/08/2026
14	4-Two elements sum to x	27/08/2026
15	5-Implementation of quick sort	27/08/2026
16	1-DP-Playing with numbers	03/09/2026
17	2-DP-Playing with chessboard	03/09/2026
18	3-DP-Longest common subsequence	10/09/2026
19	4-DP-Longest nondecreasing subsequence	10/09/2026
20	1-Finding duplicates	17/09/2026
21	2-Finding duplicates	17/09/2026
22	3-Print intersection of 2 sorted arrays	01/10/2026
23	4-Print intersection of 2 sorted arrays	01/10/2026
24	5-Pair with difference	08/10/2026
25	6-Pair with difference	08/10/2026

Finding Time Complexity of Algorithms

CS23331-DAA-2025-CSE / Problem 1: Finding Complexity using Counter Method

Problem 1: Finding Complexity using Counter Method

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;

    int s =1;

    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

```c
#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int i = 1;
    int s = 1;
    int count = 2;

    while (1)
    {
        count++;

        if (s > n)
            break;

        i++;
        count++;

        s += i;
        count++;
    }

    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Problem 2: Finding Complexity using Counter method

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int result = 5 * n + 2;

    printf("%d", result);

    return 0;
}
```

	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

Problem 3: Finding Complexity using Counter Method

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```c
#include <stdio.h>

int main()
{
    int num, i, count = 0;
    scanf("%d", &num);

    count = num + 1;

    for(i = 1; i <= num; i++)
    {
        count++;

        if(num % i == 0)
        {
            count++;
        }
    }

    printf("%d", count);

    return 0;
}
```

	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Correct

Problem 4: Finding Complexity using Counter Method

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

```c
#include <stdio.h>

int main()
{
    int n, c = 0, count = 0;
    scanf("%d", &n);

    int i = n / 2;
    count++;

    while (1)
    {
        count++;
        if (i >= n)
            break;

        int j = 1;

        while (1)
        {
            count++;
            if (j >= n)
                break;

            int k = 1;

            while (1)
            {
                count++;
                if (k >= n)
                    break;

                c++;
                count++;

                k = k * 2;
            }

            j = j * 2;
        }

        i++;
    }

    printf("%d", count);

    return 0;
}
```

	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Problem 5: Finding Complexity using counter method

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```c
#include <stdio.h>

int main()
{
    int n, rev = 0, remainder, count = 0;
    scanf("%d", &n);

    count++;

    while(n != 0)
    {
        count++;
        remainder = n % 10;
        count++;
        rev = rev * 10 + remainder;
        count++;
        n /= 10;
        count++;
    }

    count += 2;

    printf("%d", count);

    return 0;
}
```

Check

	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

Greedy Algorithms

CS23331-DAA-2025-CSE / 1-G-Coin Problem

1-G-Coin Problem

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main()
{
    int v, count = 0;
    int notes[] = {1000, 500, 100, 50, 20, 10, 5, 2,

    scanf("%d", &v);

    for(int i = 0; i < 9; i++)
    {
        count += v / notes[i];
        v = v % notes[i];
    }

    printf("%d", count);

    return 0;
}
```

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

CS23331-DAA-2025-CSE / 2-Majority Element

2-Majority Element

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array `nums` of size `n`, return *the majority element.*

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

```c

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int candidate = 0;
    int count = 0;
    int x;

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &x);

        if (count == 0)
        {
            candidate = x;
            count = 1;
        }
        else if (x == candidate)
        {
            count++;
        }
        else
        {
            count--;
        }
    }

    printf("%d", candidate);

    return 0;
}


```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

3-G-Burger Problem

Question 1 | Incorrect Mark 0.00 out of 1.00 Flag question

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories. If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$. But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

```c
#include <stdio.h>
#include <stdlib.h>


int compare(const void *a, const void *b) {
    int valA = *(const int*)a;
    int valB = *(const int*)b;

    if (valB > valA) return 1;
    if (valB < valA) return -1;
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;


    int *calories = (int *)malloc(n * sizeof(int));
    if (calories == NULL) return 1;

    for (int i = 0; i < n; i++) {
        scanf("%d", &calories[i]);
    }


    qsort(calories, n, sizeof(int), compare);

    long long total_distance = 0;
    long long power_of_three = 1;

    for (int i = 0; i < n; i++) {
        total_distance += power_of_three * calories[i];
        power_of_three *= 3;
    }

    printf("%lld\n", total_distance);


    free(calories);

    return 0;
}

```

4-G-Array Sum max problem

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return -1;
    else if (x > y)
        return 1;
    return 0;
}

int main()
{
    int n;
    scanf("%d", &n);

    int *arr = (int *)malloc(n * sizeof(int));

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &arr[i]);
    }

```

```
28
29      qsort(arr, n, sizeof(int), compare);
30
31      long long sum = 0;
32
33      for (int i = 0; i < n; i++)
34      {
35          sum += (long long)arr[i] * i;
36      }
37
38      printf("%lld", sum);
39
40      free(arr);
41
42      return 0;
43  }
44
```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

5-G-Product of Array elements-Minimum

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

```c
#include <stdio.h>
#include <stdlib.h>

int ascending(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return -1;
    if (x > y)
        return 1;
    return 0;
}

int descending(const void *a, const void *b)
{
    int x = *(const int *)a;
    int y = *(const int *)b;

    if (x < y)
        return 1;
    if (x > y)
        return -1;
    return 0;
}

int main()
{
    int n;
    scanf("%d", &n);

    long long *a = malloc(n * sizeof(long long));
    long long *b = malloc(n * sizeof(long long));

```

```
36      for (int i = 0; i < n; i++)
37          scanf("%lld", &a[i]);
38
39      for (int i = 0; i < n; i++)
40          scanf("%lld", &b[i]);
41
42      qsort(a, n, sizeof(long long), ascending);
43      qsort(b, n, sizeof(long long), descending);
44
45      long long sum = 0;
46
47      for (int i = 0; i < n; i++)
48          sum += a[i] * b[i];
49
50      printf("%lld", sum);
51
52      free(a);
53      free(b);
54
55      return 0;
56  }
57
58
```

	Input	Expected	Got	
✔	3 1 2 3 4 5 6	28	28	✔
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Divide and Conquer

vV

1-Number of Zeros in a Given Array

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

```
#include <stdio.h>

int firstZero(int arr[], int low, int high)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;


    if (arr[mid] == 0 && (mid == 0 || arr[mid - 1] == 1))
        return mid;


    if (arr[mid] == 1)
        return firstZero(arr, mid + 1, high);
```

```
    return firstZero(arr, low, mid - 1);
}

int main()
{
    int m;
    scanf("%d", &m);

    int arr[m];

    for (int i = 0; i < m; i++)
    {
        scanf("%d", &arr[i]);
    }

    int index = firstZero(arr, 0, m - 1);

    if (index == -1)
        printf("0");
    else
        printf("%d", m - index);

    return 0;
}
```

	Input	Expected	Got	
✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0	8	8	✔

2-Majority Element

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

```c

#include <stdio.h>

int main()
{
    int n;
    scanf("%d", &n);

    int candidate = 0;
    int count = 0;
    int x;

    for (int i = 0; i < n; i++)
    {
        scanf("%d", &x);

        if (count == 0)
        {
            candidate = x;
            count = 1;
        }
        else if (x == candidate)
        {
            count++;
        }
        else
        {
            count--;
        }
    }

    printf("%d", candidate);

    return 0;
}

```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

3-Finding Floor Value

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

```c

#include <stdio.h>

int findFloor(int arr[], int low, int high, int x)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;


    if (arr[mid] == x)
        return arr[mid];


    if (arr[mid] > x)
        return findFloor(arr, low, mid - 1, x);


    int result = findFloor(arr, mid + 1, high, x);

    if (result != -1)
        return result;

    return arr[mid];
}

```

```c
int main()
{
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    int x;
    scanf("%d", &x);

    int floor = findFloor(arr, 0, n - 1, x);

    printf("%d", floor);

    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔

4-Two Elements sum to x

v

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

```c
#include <stdio.h>

int binarySearch(int arr[], int low, int high, int target, int skip)
{
    if (low > high)
        return -1;

    int mid = low + (high - low) / 2;

    if (arr[mid] == target && mid != skip)
        return mid;

    if (arr[mid] < target)
        return binarySearch(arr, mid + 1, high, target, skip);

    return binarySearch(arr, low, mid - 1, target, skip);
}

int findPair(int arr[], int low, int high, int x, int n)
{
    if (low > high)
        return 0;

    int target = x - arr[low];

    int index = binarySearch(arr, 0, n - 1, target, low);
```

```
28
29      if (index != -1)
30      {
31          printf("%d\n", arr[low]);
32          printf("%d\n", arr[index]);
33          return 1;
34      }
35
36      return findPair(arr, low + 1, high, x, n);
37  }
38
39  int main()
40  {
41      int n;
42      scanf("%d", &n);
43
44      int arr[n];
45
46      for (int i = 0; i < n; i++)
47          scanf("%d", &arr[i]);
48
49      int x;
50      scanf("%d", &x);
51
52      if (!findPair(arr, 0, n - 1, x, n))
53          printf("No");
54
55      return 0;
56  }
57
```

	Input	Expected	Got	
✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

5-Implementation of Quick Sort

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

```c
#include <stdio.h>

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high];
        int i = low - 1;
        int temp;

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int p = i + 1;

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
```

```c
int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1)
            printf(" ");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Dynamic Programming

1-DP-Playing with Numbers

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    long long dp[n + 1];

    dp[0] = 1;

    for (int i = 1; i <= n; i++) {
        dp[i] = dp[i - 1];

        if (i >= 3)
            dp[i] += dp[i - 3];
    }

    printf("%lld", dp[n]);

    return 0;
}
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

2-DP-Playing with chessboard

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n][n];
    int dp[n][n];


    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &a[i][j]);
        }
    }


    dp[0][0] = a[0][0];


    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + a[0][j];
    }


    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + a[i][0];
    }
```

```
    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + a[i][0];
    }


    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (dp[i - 1][j] > dp[i][j - 1])
                dp[i][j] = dp[i - 1][j] + a[i][j];
            else
                dp[i][j] = dp[i][j - 1] + a[i][j];
        }
    }

    printf("%d\n", dp[n - 1][n - 1]);

    return 0;
}

```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

3-DP-Longest Common Subsequence

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1		a	g	**g**	**t**	**a**	**b**
s2	**g**	x	**t**	**x**	**a**	**y**	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

```c
#include <stdio.h>
#include <string.h>

int main() {
    char s1[1000], s2[1000];

    scanf("%s", s1);
    scanf("%s", s2);

    int n = strlen(s1);
    int m = strlen(s2);

    int dp[n + 1][m + 1];

    for (int i = 0; i <= n; i++)
        dp[i][0] = 0;

    for (int j = 0; j <= m; j++)
        dp[0][j] = 0;

    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (s1[i - 1] == s2[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else
                dp[i][j] = (dp[i - 1][j] > dp[i][j - 1])
                        ? dp[i - 1][j]
                        : dp[i][j - 1];
        }
    }

    printf("%d\n", dp[n][m]);

    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

4-DP-Longest non-decreasing Subsequence

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

```c
#include <stdio.h>

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high];
        int i = low - 1;
        int temp;

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;

        int p = i + 1;

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
```

```c
int main() {
    int n;
    scanf("%d", &n);

    int arr[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d", arr[i]);
        if (i < n - 1)
            printf(" ");
    }

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Competitive Programming

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d\n", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

- The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

```c
#include <stdio.h>

void findIntersection(int arr1[], int n1, int arr2[], int n2) {
    int i = 0, j = 0;
    int first = 1;

    while (i < n1 && j < n2) {
        if (arr1[i] < arr2[j]) {
            i++;
        } else if (arr1[i] > arr2[j]) {
            j++;
        } else {

            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        }
    }
    printf("\n");
}

int main() {
    int T;
    if (scanf("%d", &T) != 1) return 0;

    while (T--) {
        int n1;
        scanf("%d", &n1);
        int arr1[n1];
        for (int i = 0; i < n1; i++) {
            scanf("%d", &arr1[i]);
        }

        int n2;
        scanf("%d", &n2);
        int arr2[n2];
        for (int i = 0; i < n2; i++) {
            scanf("%d", &arr2[i]);
        }

        findIntersection(arr1, n1, arr2, n2);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

```c
#include <stdio.h>

void printIntersection(int arr1[], int n1, int arr2[], int n2) {
    int i = 0, j = 0;
    int first = 1;

    while (i < n1 && j < n2) {
        if (arr1[i] < arr2[j]) {
            i++;
        } else if (arr1[i] > arr2[j]) {
            j++;
        } else {

            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        }
    }
    printf("\n");
}
```

```c
int main() {
    int T;

    if (scanf("%d", &T) != 1) return 0;

    while (T--) {
        int n1;
        if (scanf("%d", &n1) != 1) break;
        int arr1[n1];
        for (int i = 0; i < n1; i++) {
            scanf("%d", &arr1[i]);
        }

        int n2;
        if (scanf("%d", &n2) != 1) break;
        int arr2[n2];
        for (int i = 0; i < n2; i++) {
            scanf("%d", &arr2[i]);
        }

        printIntersection(arr1, n1, arr2, n2);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

```c
#include <stdio.h>

int hasPairWithDifference(int arr[], int n, int k) {
    int i = 0;
    int j = 1;


    while (i < n && j < n) {
        int diff = arr[j] - arr[i];


        if (diff == k && i != j) {
            return 1;
        }

        else if (diff < k) {
            j++;
        }

        else {
            i++;
        }
    }

    return 0;
}
```

```c
int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int k;
    if (scanf("%d", &k) != 1) return 0;


    printf("%d\n", hasPairWithDifference(arr, n, k));

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int k;
    if (scanf("%d", &k) != 1) return 0;

    int i = 0;
    int j = 1;
    int found = 0;


    while (i < n && j < n) {
        int diff = arr[j] - arr[i];

        if (diff == k && i != j) {
            found = 1;
            break;
        } else if (diff < k) {
            j++;
        } else {
            i++;
            if (i == j) {
                j++;
            }
        }
    }


    printf("%d\n", found);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔